

Unidad II: Tuberías con el paquete magrittr

Isaac Alexander Morales Arèvalo

Contents

Unidad II: Tuberías con el paquete magrittr	2
1. Cargar paquetes	3
2. ¿Qué es una tubería?	3
2.1 Ejemplo básico con números	4
2.2 Ejemplo básico con varias funciones	4
2.3 Ejemplo básico con una base de datos	5
3. Bases de datos que usaremos	6
4. Alternativas a tuberías	7
5. Alternativa 1: guardar pasos intermedios	7
5.1 Ejemplo 1: pasos intermedios con <code>mtcars</code>	8
5.2 Ejemplo 2: pasos intermedios con <code>iris</code>	9
5.3 Ejemplo 3: pasos intermedios con <code>airquality</code>	11
6. Alternativa 2: sobrescribir el objeto original	13
6.1 Ejemplo 1: sobrescribir con <code>mtcars</code>	14
6.2 Ejemplo 2: sobrescribir con <code>iris</code>	14
6.3 Ejemplo 3: sobrescribir con <code>ToothGrowth</code>	16
7. Alternativa 3: composición de funciones	17
7.1 Ejemplo 1: composición de funciones con <code>mtcars</code>	18
7.2 Ejemplo 2: composición de funciones con <code>iris</code>	19
7.3 Ejemplo 3: composición de funciones con <code>airquality</code>	20
8. Alternativa 4: usar tuberías	23
8.1 Ejemplo 1: tuberías con <code>mtcars</code>	24
8.2 Ejemplo 2: tuberías con <code>iris</code>	24
8.3 Ejemplo 3: tuberías con <code>airquality</code>	26

9. Comparación de las cuatro formas	28
9.1 Forma 1: pasos intermedios	29
9.2 Forma 2: sobrescribir el objeto original	31
9.3 Forma 3: composición de funciones	32
9.4 Forma 4: tuberías	34
10. ¿Cómo funciona la tubería?	36
10.1 Ejemplo 1: sin tubería	38
10.2 Ejemplo 2: con tubería	39
11. Funciones que usaremos con tuberías	40
12. Ejemplo completo de clase	40
14. Ejercicios para practicar el primer ítem	41
Ejercicio 1	41
Ejercicio 2	42
Ejercicio 3	44
Ejercicio 4	45
Ejercicio 5	48
Ejercicio 6	48
Ejercicio 7	49
Ejercicio 8	50
Ejercicio 9	50
Ejercicio 10	51
16. Comentario final	52

Unidad II: Tuberías con el paquete `magrittr`

En esta unidad aprenderemos a utilizar las **tuberías** en R usando el paquete `magrittr`.

Las tuberías nos permiten escribir código de una forma más clara, ordenada y fácil de leer. Son especialmente útiles cuando queremos aplicar varias operaciones consecutivas sobre una base de datos.

El operador principal que estudiaremos es:

```
%>%
```

Este operador se puede leer como:

“luego”, “después”, “pásale esto a la siguiente función”.

Por ejemplo, el siguiente código:

```
x %>% f()
```

se interpreta como:

```
f(x)
```

Es decir, el objeto que está a la izquierda de %>% se pasa como argumento a la función que está a la derecha.

1. Cargar paquetes

Primero cargamos el paquete `magrittr`.

```
library(magrittr)
```

Si no tienes instalado el paquete, puedes instalarlo con:

```
install.packages("magrittr")
```

También usaremos el paquete `dplyr`, porque muchas funciones de manipulación de datos se entienden mejor usando tuberías.

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.4.3
```

```
##
```

```
## Adjuntando el paquete: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

Si no tienes instalado `dplyr`, puedes instalarlo con:

```
install.packages("dplyr")
```

2. ¿Qué es una tubería?

Una tubería permite tomar un objeto y enviarlo directamente a una función.

Por ejemplo:

```
sqrt(25)
```

```
## [1] 5
```

También podemos escribirlo usando tubería:

```
25 %>% sqrt()
```

```
## [1] 5
```

Ambos códigos hacen lo mismo.

La diferencia es la forma de leerlos.

El primer código se lee como:

Calcula la raíz cuadrada de 25.

El segundo código se puede leer como:

Toma 25 y luego aplícale la función raíz cuadrada.

2.1 Ejemplo básico con números

```
16 %>% sqrt()
```

```
## [1] 4
```

Este código toma el número 16 y lo pasa a la función `sqrt()`.

Es equivalente a:

```
sqrt(16)
```

```
## [1] 4
```

2.2 Ejemplo básico con varias funciones

```
25 %>%  
  sqrt() %>%  
  log()
```

```
## [1] 1.609438
```

Este código hace lo siguiente:

1. Toma el número 25.
2. Calcula su raíz cuadrada.
3. Al resultado le aplica la función logaritmo.

Es equivalente a:

```
log(sqrt(25))
```

```
## [1] 1.609438
```

La ventaja de la tubería es que podemos leer el código de arriba hacia abajo.

2.3 Ejemplo básico con una base de datos

```
data(mtcars)
```

```
mtcars %>%  
  head()
```

```
##           mpg cyl  disp  hp  drat    wt  qsec vs am gear carb  
## Mazda RX4      21.0   6  160 110 3.90 2.620 16.46 0  1   4    4  
## Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02 0  1   4    4  
## Datsun 710      22.8   4  108  93 3.85 2.320 18.61 1  1   4    1  
## Hornet 4 Drive  21.4   6  258 110 3.08 3.215 19.44 1  0   3    1  
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02 0  0   3    2  
## Valiant        18.1   6  225 105 2.76 3.460 20.22 1  0   3    1
```

Aquí tomamos la base de datos `mtcars` y la pasamos a la función `head()` para ver las primeras filas.

Es equivalente a:

```
head(mtcars)
```

```
##           mpg cyl  disp  hp  drat    wt  qsec vs am gear carb  
## Mazda RX4      21.0   6  160 110 3.90 2.620 16.46 0  1   4    4  
## Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02 0  1   4    4  
## Datsun 710      22.8   4  108  93 3.85 2.320 18.61 1  1   4    1  
## Hornet 4 Drive  21.4   6  258 110 3.08 3.215 19.44 1  0   3    1  
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02 0  0   3    2  
## Valiant        18.1   6  225 105 2.76 3.460 20.22 1  0   3    1
```

3. Bases de datos que usaremos

En esta clase usaremos algunas bases de datos que ya vienen incluidas en R.

Algunas de ellas son:

- `mtcars`: información sobre automóviles.
- `iris`: información sobre flores.
- `airquality`: información sobre calidad del aire.
- `ToothGrowth`: crecimiento dental en conejillos de indias.

Veamos la base `mtcars`.

```
data(mtcars)
head(mtcars)
```

```
##           mpg  cyl  disp  hp  drat    wt   qsec vs  am  gear  carb
## Mazda RX4      21.0   6  160 110 3.90 2.620 16.46 0   1    4    4
## Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02 0   1    4    4
## Datsun 710      22.8   4  108  93 3.85 2.320 18.61 1   1    4    1
## Hornet 4 Drive  21.4   6  258 110 3.08 3.215 19.44 1   0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02 0   0    3    2
## Valiant        18.1   6  225 105 2.76 3.460 20.22 1   0    3    1
```

Algunas variables importantes de `mtcars` son:

- `mpg`: millas por galón.
- `cyl`: número de cilindros.
- `disp`: desplazamiento del motor.
- `hp`: caballos de fuerza.
- `wt`: peso del automóvil.
- `gear`: número de marchas.

Veamos también la base `iris`.

```
data(iris)
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1         5.1         3.5          1.4          0.2   setosa
## 2         4.9         3.0          1.4          0.2   setosa
## 3         4.7         3.2          1.3          0.2   setosa
## 4         4.6         3.1          1.5          0.2   setosa
## 5         5.0         3.6          1.4          0.2   setosa
## 6         5.4         3.9          1.7          0.4   setosa
```

Algunas variables importantes de `iris` son:

- `Sepal.Length`: longitud del sépalo.
- `Sepal.Width`: ancho del sépalo.
- `Petal.Length`: longitud del pétalo.
- `Petal.Width`: ancho del pétalo.
- `Species`: especie de la flor.

4. Alternativas a tuberías

Antes de usar tuberías, es importante conocer otras formas de escribir una secuencia de operaciones.

En esta clase estudiaremos cuatro alternativas:

1. Guardar pasos intermedios.
2. Sobrescribir el objeto original.
3. Componer funciones.
4. Usar tuberías.

La idea es comparar estas formas y entender por qué las tuberías pueden hacer que el código sea más claro.

Supongamos que queremos hacer lo siguiente:

1. Tomar una base de datos.
2. Seleccionar algunas columnas.
3. Filtrar algunas filas.
4. Ordenar los resultados.

Esto se puede hacer de varias maneras.

5. Alternativa 1: guardar pasos intermedios

La primera forma consiste en guardar cada paso en un objeto diferente.

Por ejemplo:

```
paso1 <- mtcars
paso2 <- select(paso1, mpg, cyl, hp, wt)
paso3 <- filter(paso2, hp > 100)
paso4 <- arrange(paso3, desc(mpg))

paso4
```

##	mpg	cyl	hp	wt
## Lotus Europa	30.4	4	113	1.513
## Hornet 4 Drive	21.4	6	110	3.215
## Volvo 142E	21.4	4	109	2.780
## Mazda RX4	21.0	6	110	2.620
## Mazda RX4 Wag	21.0	6	110	2.875
## Ferrari Dino	19.7	6	175	2.770
## Merc 280	19.2	6	123	3.440
## Pontiac Firebird	19.2	8	175	3.845
## Hornet Sportabout	18.7	8	175	3.440
## Valiant	18.1	6	105	3.460
## Merc 280C	17.8	6	123	3.440
## Merc 450SL	17.3	8	180	3.730
## Merc 450SE	16.4	8	180	4.070
## Ford Pantera L	15.8	8	264	3.170

```
## Dodge Challenger      15.5   8 150 3.520
## Merc 450SLC          15.2   8 180 3.780
## AMC Javelin          15.2   8 150 3.435
## Maserati Bora        15.0   8 335 3.570
## Chrysler Imperial    14.7   8 230 5.345
## Duster 360           14.3   8 245 3.570
## Camaro Z28           13.3   8 245 3.840
## Cadillac Fleetwood   10.4   8 205 5.250
## Lincoln Continental  10.4   8 215 5.424
```

Aquí hicimos lo siguiente:

1. Guardamos la base `mtcars` en `paso1`.
2. Seleccionamos algunas columnas y guardamos el resultado en `paso2`.
3. Filtramos automóviles con más de 100 caballos de fuerza y guardamos el resultado en `paso3`.
4. Ordenamos los resultados de mayor a menor según `mpg` y guardamos el resultado en `paso4`.

Esta forma es clara porque podemos revisar cada paso.

Sin embargo, tiene una desventaja: necesitamos crear muchos nombres, como `paso1`, `paso2`, `paso3`, etc.

5.1 Ejemplo 1: pasos intermedios con `mtcars`

Seleccionemos algunas columnas, filtremos automóviles con más de 100 caballos de fuerza y ordenemos por rendimiento.

```
autos1 <- mtcars
autos2 <- select(autos1, mpg, cyl, hp, wt)
autos3 <- filter(autos2, hp > 100)
autos4 <- arrange(autos3, desc(mpg))

autos4
```

```
##           mpg cyl  hp  wt
## Lotus Europa    30.4   4 113 1.513
## Hornet 4 Drive  21.4   6 110 3.215
## Volvo 142E      21.4   4 109 2.780
## Mazda RX4       21.0   6 110 2.620
## Mazda RX4 Wag   21.0   6 110 2.875
## Ferrari Dino    19.7   6 175 2.770
## Merc 280         19.2   6 123 3.440
## Pontiac Firebird 19.2   8 175 3.845
## Hornet Sportabout 18.7   8 175 3.440
## Valiant         18.1   6 105 3.460
## Merc 280C        17.8   6 123 3.440
## Merc 450SL       17.3   8 180 3.730
## Merc 450SE       16.4   8 180 4.070
## Ford Pantera L   15.8   8 264 3.170
## Dodge Challenger 15.5   8 150 3.520
## Merc 450SLC      15.2   8 180 3.780
```

```
## AMC Javelin      15.2   8 150 3.435
## Maserati Bora    15.0   8 335 3.570
## Chrysler Imperial 14.7   8 230 5.345
## Duster 360      14.3   8 245 3.570
## Camaro Z28       13.3   8 245 3.840
## Cadillac Fleetwood 10.4   8 205 5.250
## Lincoln Continental 10.4   8 215 5.424
```

5.2 Ejemplo 2: pasos intermedios con iris

Seleccionemos algunas columnas, filtremos flores con pétalos largos y ordenemos por longitud del pétalo.

```
flores1 <- iris
flores2 <- select(flores1, Sepal.Length, Petal.Length, Species)
flores3 <- filter(flores2, Petal.Length > 4)
flores4 <- arrange(flores3, desc(Petal.Length))

flores4
```

```
##   Sepal.Length Petal.Length   Species
## 1         7.7         6.9 virginica
## 2         7.7         6.7 virginica
## 3         7.7         6.7 virginica
## 4         7.6         6.6 virginica
## 5         7.9         6.4 virginica
## 6         7.3         6.3 virginica
## 7         7.2         6.1 virginica
## 8         7.4         6.1 virginica
## 9         7.7         6.1 virginica
## 10        6.3         6.0 virginica
## 11        7.2         6.0 virginica
## 12        7.1         5.9 virginica
## 13        6.8         5.9 virginica
## 14        6.5         5.8 virginica
## 15        6.7         5.8 virginica
## 16        7.2         5.8 virginica
## 17        6.9         5.7 virginica
## 18        6.7         5.7 virginica
## 19        6.7         5.7 virginica
## 20        6.3         5.6 virginica
## 21        6.4         5.6 virginica
## 22        6.4         5.6 virginica
## 23        6.1         5.6 virginica
## 24        6.3         5.6 virginica
## 25        6.7         5.6 virginica
## 26        6.8         5.5 virginica
## 27        6.5         5.5 virginica
## 28        6.4         5.5 virginica
## 29        6.9         5.4 virginica
## 30        6.2         5.4 virginica
## 31        6.4         5.3 virginica
```

## 32	6.4	5.3 virginica
## 33	6.7	5.2 virginica
## 34	6.5	5.2 virginica
## 35	6.0	5.1 versicolor
## 36	5.8	5.1 virginica
## 37	6.5	5.1 virginica
## 38	5.8	5.1 virginica
## 39	6.3	5.1 virginica
## 40	6.9	5.1 virginica
## 41	5.8	5.1 virginica
## 42	5.9	5.1 virginica
## 43	6.7	5.0 versicolor
## 44	5.7	5.0 virginica
## 45	6.0	5.0 virginica
## 46	6.3	5.0 virginica
## 47	6.9	4.9 versicolor
## 48	6.3	4.9 versicolor
## 49	5.6	4.9 virginica
## 50	6.3	4.9 virginica
## 51	6.1	4.9 virginica
## 52	5.9	4.8 versicolor
## 53	6.8	4.8 versicolor
## 54	6.2	4.8 virginica
## 55	6.0	4.8 virginica
## 56	7.0	4.7 versicolor
## 57	6.3	4.7 versicolor
## 58	6.1	4.7 versicolor
## 59	6.1	4.7 versicolor
## 60	6.7	4.7 versicolor
## 61	6.5	4.6 versicolor
## 62	6.6	4.6 versicolor
## 63	6.1	4.6 versicolor
## 64	6.4	4.5 versicolor
## 65	5.7	4.5 versicolor
## 66	5.6	4.5 versicolor
## 67	6.2	4.5 versicolor
## 68	6.0	4.5 versicolor
## 69	5.4	4.5 versicolor
## 70	6.0	4.5 versicolor
## 71	4.9	4.5 virginica
## 72	6.7	4.4 versicolor
## 73	6.6	4.4 versicolor
## 74	6.3	4.4 versicolor
## 75	5.5	4.4 versicolor
## 76	6.4	4.3 versicolor
## 77	6.2	4.3 versicolor
## 78	5.9	4.2 versicolor
## 79	5.6	4.2 versicolor
## 80	5.7	4.2 versicolor
## 81	5.7	4.2 versicolor
## 82	5.8	4.1 versicolor
## 83	5.6	4.1 versicolor
## 84	5.7	4.1 versicolor

5.3 Ejemplo 3: pasos intermedios con airquality

Eliminemos valores faltantes, seleccionemos algunas columnas y ordenemos por temperatura.

```
data(airquality)

aire1 <- airquality
aire2 <- na.omit(aire1)
aire3 <- select(aire2, Ozone, Wind, Temp, Month)
aire4 <- arrange(aire3, desc(Temp))

aire4
```

##		Ozone	Wind	Temp	Month
## 1		76	9.7	97	8
## 2		84	6.3	96	8
## 3		118	2.3	94	8
## 4		85	6.3	94	8
## 5		73	2.8	93	9
## 6		91	4.6	93	9
## 7		97	6.3	92	7
## 8		97	5.7	92	7
## 9		78	5.1	92	9
## 10		96	6.9	91	9
## 11		71	13.8	90	6
## 12		89	10.3	90	8
## 13		110	8.0	90	8
## 14		85	7.4	89	7
## 15		122	4.0	89	8
## 16		77	5.1	88	7
## 17		82	7.4	88	7
## 18		39	11.5	87	6
## 19		79	5.1	87	7
## 20		47	7.4	87	9
## 21		80	8.6	86	7
## 22		52	12.0	86	7
## 23		50	7.4	86	7
## 24		44	11.5	86	8
## 25		73	8.0	86	8
## 26		49	9.2	85	7
## 27		63	11.5	85	7
## 28		108	8.0	85	7
## 29		135	4.1	84	7
## 30		61	6.3	84	7
## 31		32	15.5	84	9
## 32		64	4.6	83	7
## 33		40	10.9	83	7
## 34		64	7.4	83	7
## 35		29	9.7	82	6
## 36		23	8.0	82	6
## 37		35	10.3	82	7

## 38	20	8.6	82	7
## 39	16	7.4	82	8
## 40	28	11.5	82	8
## 41	16	8.0	82	9
## 42	45	14.9	81	5
## 43	32	9.2	81	7
## 44	27	14.9	81	7
## 45	48	6.9	81	7
## 46	59	9.2	81	7
## 47	39	6.9	81	8
## 48	9	13.8	81	8
## 49	168	3.4	81	8
## 50	44	14.9	81	9
## 51	36	10.3	81	9
## 52	7	14.3	80	7
## 53	65	9.7	80	8
## 54	20	10.9	80	9
## 55	115	5.7	79	5
## 56	59	6.3	79	8
## 57	45	9.7	79	8
## 58	31	10.9	78	8
## 59	44	10.3	78	8
## 60	23	10.3	78	9
## 61	46	6.9	78	9
## 62	21	14.9	77	6
## 63	22	10.3	77	8
## 64	21	15.5	77	8
## 65	28	6.3	77	9
## 66	37	7.4	76	5
## 67	13	10.3	76	6
## 68	23	7.4	76	8
## 69	21	15.5	76	9
## 70	13	10.3	76	9
## 71	18	8.0	76	9
## 72	21	10.9	75	9
## 73	14	14.3	75	9
## 74	12	12.6	74	5
## 75	16	6.9	74	7
## 76	11	16.6	73	5
## 77	12	11.5	73	6
## 78	10	14.3	73	7
## 79	24	9.7	73	9
## 80	36	8.0	72	5
## 81	37	20.7	72	6
## 82	9	14.3	72	8
## 83	9	10.9	71	9
## 84	13	11.5	71	9
## 85	23	9.2	71	9
## 86	30	6.9	70	9
## 87	16	9.7	69	5
## 88	7	10.3	69	9
## 89	14	10.9	68	5
## 90	30	11.5	68	5
## 91	24	10.3	68	9

```
## 92      20 11.5   68    9
## 93      41  7.4   67    5
## 94      23 12.0   67    5
## 95      18 13.8   67    9
## 96      11  9.2   66    5
## 97      34 12.0   66    5
## 98      23  8.6   65    5
## 99      20  9.2   65    6
## 100     14 11.5   64    5
## 101     13 12.6   64    9
## 102     14 16.6   63    9
## 103     18 11.5   62    5
## 104     11  9.7   62    5
## 105      8 20.1   61    5
## 106      4  9.7   61    5
## 107     32 12.0   61    5
## 108     19 13.8   59    5
## 109      1  9.7   59    5
## 110     18 13.2   58    5
## 111      6 18.4   57    5
```

6. Alternativa 2: sobrescribir el objeto original

Otra forma consiste en usar siempre el mismo nombre del objeto.

Por ejemplo:

```
autos <- mtcars
autos <- select(autos, mpg, cyl, hp, wt)
autos <- filter(autos, hp > 100)
autos <- arrange(autos, desc(mpg))

autos
```

```
##           mpg  cyl  hp   wt
## Lotus Europa    30.4   4 113 1.513
## Hornet 4 Drive  21.4   6 110 3.215
## Volvo 142E      21.4   4 109 2.780
## Mazda RX4       21.0   6 110 2.620
## Mazda RX4 Wag   21.0   6 110 2.875
## Ferrari Dino    19.7   6 175 2.770
## Merc 280        19.2   6 123 3.440
## Pontiac Firebird 19.2   8 175 3.845
## Hornet Sportabout 18.7   8 175 3.440
## Valiant         18.1   6 105 3.460
## Merc 280C       17.8   6 123 3.440
## Merc 450SL      17.3   8 180 3.730
## Merc 450SE      16.4   8 180 4.070
## Ford Pantera L  15.8   8 264 3.170
## Dodge Challenger 15.5   8 150 3.520
```

```
## Merc 450SLC      15.2   8 180 3.780
## AMC Javelin      15.2   8 150 3.435
## Maserati Bora    15.0   8 335 3.570
## Chrysler Imperial 14.7   8 230 5.345
## Duster 360       14.3   8 245 3.570
## Camaro Z28       13.3   8 245 3.840
## Cadillac Fleetwood 10.4   8 205 5.250
## Lincoln Continental 10.4   8 215 5.424
```

Aquí el objeto `autos` se va modificando en cada línea.

La ventaja es que usamos menos nombres.

La desventaja es que si cometemos un error, puede ser más difícil revisar qué ocurrió en los pasos anteriores.

6.1 Ejemplo 1: sobrescribir con `mtcars`

```
autos <- mtcars
autos <- select(autos, mpg, cyl, hp, wt)
autos <- filter(autos, cyl == 6)
autos <- arrange(autos, desc(hp))

autos
```

```
##           mpg cyl  hp  wt
## Ferrari Dino  19.7   6 175 2.770
## Merc 280      19.2   6 123 3.440
## Merc 280C     17.8   6 123 3.440
## Mazda RX4     21.0   6 110 2.620
## Mazda RX4 Wag 21.0   6 110 2.875
## Hornet 4 Drive 21.4   6 110 3.215
## Valiant       18.1   6 105 3.460
```

6.2 Ejemplo 2: sobrescribir con `iris`

```
flores <- iris
flores <- select(flores, Sepal.Length, Sepal.Width, Species)
flores <- filter(flores, Sepal.Width > 3)
flores <- arrange(flores, Species, desc(Sepal.Length))

flores
```

```
##   Sepal.Length Sepal.Width  Species
## 1          5.8         4.0   setosa
## 2          5.7         4.4   setosa
## 3          5.7         3.8   setosa
```

## 4	5.5	4.2	setosa
## 5	5.5	3.5	setosa
## 6	5.4	3.9	setosa
## 7	5.4	3.7	setosa
## 8	5.4	3.9	setosa
## 9	5.4	3.4	setosa
## 10	5.4	3.4	setosa
## 11	5.3	3.7	setosa
## 12	5.2	3.5	setosa
## 13	5.2	3.4	setosa
## 14	5.2	4.1	setosa
## 15	5.1	3.5	setosa
## 16	5.1	3.5	setosa
## 17	5.1	3.8	setosa
## 18	5.1	3.7	setosa
## 19	5.1	3.3	setosa
## 20	5.1	3.4	setosa
## 21	5.1	3.8	setosa
## 22	5.1	3.8	setosa
## 23	5.0	3.6	setosa
## 24	5.0	3.4	setosa
## 25	5.0	3.4	setosa
## 26	5.0	3.2	setosa
## 27	5.0	3.5	setosa
## 28	5.0	3.5	setosa
## 29	5.0	3.3	setosa
## 30	4.9	3.1	setosa
## 31	4.9	3.1	setosa
## 32	4.9	3.6	setosa
## 33	4.8	3.4	setosa
## 34	4.8	3.4	setosa
## 35	4.8	3.1	setosa
## 36	4.7	3.2	setosa
## 37	4.7	3.2	setosa
## 38	4.6	3.1	setosa
## 39	4.6	3.4	setosa
## 40	4.6	3.6	setosa
## 41	4.6	3.2	setosa
## 42	4.4	3.2	setosa
## 43	7.0	3.2	versicolor
## 44	6.9	3.1	versicolor
## 45	6.7	3.1	versicolor
## 46	6.7	3.1	versicolor
## 47	6.4	3.2	versicolor
## 48	6.3	3.3	versicolor
## 49	6.0	3.4	versicolor
## 50	5.9	3.2	versicolor
## 51	7.9	3.8	virginica
## 52	7.7	3.8	virginica
## 53	7.2	3.6	virginica
## 54	7.2	3.2	virginica
## 55	6.9	3.2	virginica
## 56	6.9	3.1	virginica
## 57	6.9	3.1	virginica

```
## 58      6.8      3.2 virginica
## 59      6.7      3.3 virginica
## 60      6.7      3.1 virginica
## 61      6.7      3.3 virginica
## 62      6.5      3.2 virginica
## 63      6.4      3.2 virginica
## 64      6.4      3.1 virginica
## 65      6.3      3.3 virginica
## 66      6.3      3.4 virginica
## 67      6.2      3.4 virginica
```

6.3 Ejemplo 3: sobrescribir con ToothGrowth

```
data(ToothGrowth)

dientes <- ToothGrowth
dientes <- select(dientes, len, supp, dose)
dientes <- filter(dientes, dose >= 1)
dientes <- arrange(dientes, supp, desc(len))

dientes
```

```
##      len supp dose
## 1  30.9   OJ    2
## 2  29.4   OJ    2
## 3  27.3   OJ    1
## 4  27.3   OJ    2
## 5  26.4   OJ    1
## 6  26.4   OJ    2
## 7  26.4   OJ    2
## 8  25.8   OJ    1
## 9  25.5   OJ    2
## 10 25.2   OJ    1
## 11 24.8   OJ    2
## 12 24.5   OJ    2
## 13 23.6   OJ    1
## 14 23.3   OJ    1
## 15 23.0   OJ    2
## 16 22.4   OJ    2
## 17 21.2   OJ    1
## 18 20.0   OJ    1
## 19 19.7   OJ    1
## 20 14.5   OJ    1
## 21 33.9   VC    2
## 22 32.5   VC    2
## 23 29.5   VC    2
## 24 26.7   VC    2
## 25 26.4   VC    2
## 26 25.5   VC    2
## 27 23.6   VC    2
```

```
## 28 23.3 VC 2
## 29 22.5 VC 1
## 30 21.5 VC 2
## 31 18.8 VC 1
## 32 18.5 VC 2
## 33 17.3 VC 1
## 34 17.3 VC 1
## 35 16.5 VC 1
## 36 16.5 VC 1
## 37 15.5 VC 1
## 38 15.2 VC 1
## 39 14.5 VC 1
## 40 13.6 VC 1
```

7. Alternativa 3: composición de funciones

La composición de funciones consiste en escribir una función dentro de otra función.

Por ejemplo:

```
arrange(
  filter(
    select(mtcars, mpg, cyl, hp, wt),
    hp > 100
  ),
  desc(mpg)
)
```

```
##           mpg cyl  hp  wt
## Lotus Europa    30.4  4 113 1.513
## Hornet 4 Drive  21.4  6 110 3.215
## Volvo 142E      21.4  4 109 2.780
## Mazda RX4      21.0  6 110 2.620
## Mazda RX4 Wag  21.0  6 110 2.875
## Ferrari Dino    19.7  6 175 2.770
## Merc 280        19.2  6 123 3.440
## Pontiac Firebird 19.2  8 175 3.845
## Hornet Sportabout 18.7  8 175 3.440
## Valiant         18.1  6 105 3.460
## Merc 280C       17.8  6 123 3.440
## Merc 450SL      17.3  8 180 3.730
## Merc 450SE      16.4  8 180 4.070
## Ford Pantera L  15.8  8 264 3.170
## Dodge Challenger 15.5  8 150 3.520
## Merc 450SLC     15.2  8 180 3.780
## AMC Javelin     15.2  8 150 3.435
## Maserati Bora   15.0  8 335 3.570
## Chrysler Imperial 14.7  8 230 5.345
## Duster 360      14.3  8 245 3.570
## Camaro Z28      13.3  8 245 3.840
```

```
## Cadillac Fleetwood 10.4 8 205 5.250
## Lincoln Continental 10.4 8 215 5.424
```

Este código funciona, pero puede ser más difícil de leer.

La razón es que R evalúa primero lo que está más adentro.

En este ejemplo, el orden real de ejecución es:

1. Primero se ejecuta `select(mtcars, mpg, cyl, hp, wt)`.
2. Luego se ejecuta `filter(..., hp > 100)`.
3. Finalmente se ejecuta `arrange(..., desc(mpg))`.

El problema es que el código se lee de afuera hacia adentro, pero se ejecuta de adentro hacia afuera.

7.1 Ejemplo 1: composición de funciones con `mtcars`

```
arrange(
  filter(
    select(mtcars, mpg, cyl, hp, wt),
    hp > 100
  ),
  desc(mpg)
)
```

```
##           mpg cyl  hp   wt
## Lotus Europa      30.4  4 113 1.513
## Hornet 4 Drive    21.4  6 110 3.215
## Volvo 142E        21.4  4 109 2.780
## Mazda RX4        21.0  6 110 2.620
## Mazda RX4 Wag    21.0  6 110 2.875
## Ferrari Dino      19.7  6 175 2.770
## Merc 280          19.2  6 123 3.440
## Pontiac Firebird  19.2  8 175 3.845
## Hornet Sportabout 18.7  8 175 3.440
## Valiant           18.1  6 105 3.460
## Merc 280C         17.8  6 123 3.440
## Merc 450SL        17.3  8 180 3.730
## Merc 450SE        16.4  8 180 4.070
## Ford Pantera L    15.8  8 264 3.170
## Dodge Challenger  15.5  8 150 3.520
## Merc 450SLC       15.2  8 180 3.780
## AMC Javelin       15.2  8 150 3.435
## Maserati Bora     15.0  8 335 3.570
## Chrysler Imperial 14.7  8 230 5.345
## Duster 360        14.3  8 245 3.570
## Camaro Z28        13.3  8 245 3.840
## Cadillac Fleetwood 10.4  8 205 5.250
## Lincoln Continental 10.4  8 215 5.424
```

7.2 Ejemplo 2: composición de funciones con iris

```
arrange(  
  filter(  
    select(iris, Sepal.Length, Petal.Length, Species),  
    Petal.Length > 4  
  ),  
  desc(Petal.Length)  
)
```

##	Sepal.Length	Petal.Length	Species
## 1	7.7	6.9	virginica
## 2	7.7	6.7	virginica
## 3	7.7	6.7	virginica
## 4	7.6	6.6	virginica
## 5	7.9	6.4	virginica
## 6	7.3	6.3	virginica
## 7	7.2	6.1	virginica
## 8	7.4	6.1	virginica
## 9	7.7	6.1	virginica
## 10	6.3	6.0	virginica
## 11	7.2	6.0	virginica
## 12	7.1	5.9	virginica
## 13	6.8	5.9	virginica
## 14	6.5	5.8	virginica
## 15	6.7	5.8	virginica
## 16	7.2	5.8	virginica
## 17	6.9	5.7	virginica
## 18	6.7	5.7	virginica
## 19	6.7	5.7	virginica
## 20	6.3	5.6	virginica
## 21	6.4	5.6	virginica
## 22	6.4	5.6	virginica
## 23	6.1	5.6	virginica
## 24	6.3	5.6	virginica
## 25	6.7	5.6	virginica
## 26	6.8	5.5	virginica
## 27	6.5	5.5	virginica
## 28	6.4	5.5	virginica
## 29	6.9	5.4	virginica
## 30	6.2	5.4	virginica
## 31	6.4	5.3	virginica
## 32	6.4	5.3	virginica
## 33	6.7	5.2	virginica
## 34	6.5	5.2	virginica
## 35	6.0	5.1	versicolor
## 36	5.8	5.1	virginica
## 37	6.5	5.1	virginica
## 38	5.8	5.1	virginica
## 39	6.3	5.1	virginica
## 40	6.9	5.1	virginica
## 41	5.8	5.1	virginica
## 42	5.9	5.1	virginica

## 43	6.7	5.0 versicolor
## 44	5.7	5.0 virginica
## 45	6.0	5.0 virginica
## 46	6.3	5.0 virginica
## 47	6.9	4.9 versicolor
## 48	6.3	4.9 versicolor
## 49	5.6	4.9 virginica
## 50	6.3	4.9 virginica
## 51	6.1	4.9 virginica
## 52	5.9	4.8 versicolor
## 53	6.8	4.8 versicolor
## 54	6.2	4.8 virginica
## 55	6.0	4.8 virginica
## 56	7.0	4.7 versicolor
## 57	6.3	4.7 versicolor
## 58	6.1	4.7 versicolor
## 59	6.1	4.7 versicolor
## 60	6.7	4.7 versicolor
## 61	6.5	4.6 versicolor
## 62	6.6	4.6 versicolor
## 63	6.1	4.6 versicolor
## 64	6.4	4.5 versicolor
## 65	5.7	4.5 versicolor
## 66	5.6	4.5 versicolor
## 67	6.2	4.5 versicolor
## 68	6.0	4.5 versicolor
## 69	5.4	4.5 versicolor
## 70	6.0	4.5 versicolor
## 71	4.9	4.5 virginica
## 72	6.7	4.4 versicolor
## 73	6.6	4.4 versicolor
## 74	6.3	4.4 versicolor
## 75	5.5	4.4 versicolor
## 76	6.4	4.3 versicolor
## 77	6.2	4.3 versicolor
## 78	5.9	4.2 versicolor
## 79	5.6	4.2 versicolor
## 80	5.7	4.2 versicolor
## 81	5.7	4.2 versicolor
## 82	5.8	4.1 versicolor
## 83	5.6	4.1 versicolor
## 84	5.7	4.1 versicolor

7.3 Ejemplo 3: composición de funciones con airquality

```

arrange(
  select(
    na.omit(airquality),
    Ozone, Wind, Temp, Month
  ),

```

```
desc(Temp)
)
```

##	Ozone	Wind	Temp	Month
## 1	76	9.7	97	8
## 2	84	6.3	96	8
## 3	118	2.3	94	8
## 4	85	6.3	94	8
## 5	73	2.8	93	9
## 6	91	4.6	93	9
## 7	97	6.3	92	7
## 8	97	5.7	92	7
## 9	78	5.1	92	9
## 10	96	6.9	91	9
## 11	71	13.8	90	6
## 12	89	10.3	90	8
## 13	110	8.0	90	8
## 14	85	7.4	89	7
## 15	122	4.0	89	8
## 16	77	5.1	88	7
## 17	82	7.4	88	7
## 18	39	11.5	87	6
## 19	79	5.1	87	7
## 20	47	7.4	87	9
## 21	80	8.6	86	7
## 22	52	12.0	86	7
## 23	50	7.4	86	7
## 24	44	11.5	86	8
## 25	73	8.0	86	8
## 26	49	9.2	85	7
## 27	63	11.5	85	7
## 28	108	8.0	85	7
## 29	135	4.1	84	7
## 30	61	6.3	84	7
## 31	32	15.5	84	9
## 32	64	4.6	83	7
## 33	40	10.9	83	7
## 34	64	7.4	83	7
## 35	29	9.7	82	6
## 36	23	8.0	82	6
## 37	35	10.3	82	7
## 38	20	8.6	82	7
## 39	16	7.4	82	8
## 40	28	11.5	82	8
## 41	16	8.0	82	9
## 42	45	14.9	81	5
## 43	32	9.2	81	7
## 44	27	14.9	81	7
## 45	48	6.9	81	7
## 46	59	9.2	81	7
## 47	39	6.9	81	8
## 48	9	13.8	81	8
## 49	168	3.4	81	8

## 50	44	14.9	81	9
## 51	36	10.3	81	9
## 52	7	14.3	80	7
## 53	65	9.7	80	8
## 54	20	10.9	80	9
## 55	115	5.7	79	5
## 56	59	6.3	79	8
## 57	45	9.7	79	8
## 58	31	10.9	78	8
## 59	44	10.3	78	8
## 60	23	10.3	78	9
## 61	46	6.9	78	9
## 62	21	14.9	77	6
## 63	22	10.3	77	8
## 64	21	15.5	77	8
## 65	28	6.3	77	9
## 66	37	7.4	76	5
## 67	13	10.3	76	6
## 68	23	7.4	76	8
## 69	21	15.5	76	9
## 70	13	10.3	76	9
## 71	18	8.0	76	9
## 72	21	10.9	75	9
## 73	14	14.3	75	9
## 74	12	12.6	74	5
## 75	16	6.9	74	7
## 76	11	16.6	73	5
## 77	12	11.5	73	6
## 78	10	14.3	73	7
## 79	24	9.7	73	9
## 80	36	8.0	72	5
## 81	37	20.7	72	6
## 82	9	14.3	72	8
## 83	9	10.9	71	9
## 84	13	11.5	71	9
## 85	23	9.2	71	9
## 86	30	6.9	70	9
## 87	16	9.7	69	5
## 88	7	10.3	69	9
## 89	14	10.9	68	5
## 90	30	11.5	68	5
## 91	24	10.3	68	9
## 92	20	11.5	68	9
## 93	41	7.4	67	5
## 94	23	12.0	67	5
## 95	18	13.8	67	9
## 96	11	9.2	66	5
## 97	34	12.0	66	5
## 98	23	8.6	65	5
## 99	20	9.2	65	6
## 100	14	11.5	64	5
## 101	13	12.6	64	9
## 102	14	16.6	63	9
## 103	18	11.5	62	5

```
## 104      11  9.7   62    5
## 105       8 20.1   61    5
## 106       4  9.7   61    5
## 107      32 12.0   61    5
## 108      19 13.8   59    5
## 109       1  9.7   59    5
## 110      18 13.2   58    5
## 111       6 18.4   57    5
```

8. Alternativa 4: usar tuberías

La forma con tuberías permite escribir las operaciones en el mismo orden en que las pensamos.

Por ejemplo:

```
mtcars %>%
  select(mpg, cyl, hp, wt) %>%
  filter(hp > 100) %>%
  arrange(desc(mpg))
```

```
##              mpg cyl  hp   wt
## Lotus Europa    30.4   4 113 1.513
## Hornet 4 Drive  21.4   6 110 3.215
## Volvo 142E      21.4   4 109 2.780
## Mazda RX4      21.0   6 110 2.620
## Mazda RX4 Wag  21.0   6 110 2.875
## Ferrari Dino    19.7   6 175 2.770
## Merc 280        19.2   6 123 3.440
## Pontiac Firebird 19.2   8 175 3.845
## Hornet Sportabout 18.7   8 175 3.440
## Valiant         18.1   6 105 3.460
## Merc 280C       17.8   6 123 3.440
## Merc 450SL      17.3   8 180 3.730
## Merc 450SE      16.4   8 180 4.070
## Ford Pantera L  15.8   8 264 3.170
## Dodge Challenger 15.5   8 150 3.520
## Merc 450SLC     15.2   8 180 3.780
## AMC Javelin     15.2   8 150 3.435
## Maserati Bora   15.0   8 335 3.570
## Chrysler Imperial 14.7   8 230 5.345
## Duster 360      14.3   8 245 3.570
## Camaro Z28      13.3   8 245 3.840
## Cadillac Fleetwood 10.4   8 205 5.250
## Lincoln Continental 10.4   8 215 5.424
```

Este código se lee así:

Toma la base `mtcars`, luego selecciona las columnas `mpg`, `cyl`, `hp` y `wt`, luego filtra los automóviles con más de 100 caballos de fuerza, y luego ordénalos de mayor a menor según `mpg`.

La tubería permite que el código se lea de arriba hacia abajo.

8.1 Ejemplo 1: tuberías con mtcars

```
mtcars %>%  
  select(mpg, cyl, hp, wt) %>%  
  filter(hp > 100) %>%  
  arrange(desc(mpg))
```

```
##           mpg  cyl  hp   wt  
## Lotus Europa    30.4   4 113 1.513  
## Hornet 4 Drive  21.4   6 110 3.215  
## Volvo 142E      21.4   4 109 2.780  
## Mazda RX4       21.0   6 110 2.620  
## Mazda RX4 Wag   21.0   6 110 2.875  
## Ferrari Dino    19.7   6 175 2.770  
## Merc 280        19.2   6 123 3.440  
## Pontiac Firebird 19.2   8 175 3.845  
## Hornet Sportabout 18.7   8 175 3.440  
## Valiant         18.1   6 105 3.460  
## Merc 280C       17.8   6 123 3.440  
## Merc 450SL      17.3   8 180 3.730  
## Merc 450SE      16.4   8 180 4.070  
## Ford Pantera L  15.8   8 264 3.170  
## Dodge Challenger 15.5   8 150 3.520  
## Merc 450SLC     15.2   8 180 3.780  
## AMC Javelin     15.2   8 150 3.435  
## Maserati Bora   15.0   8 335 3.570  
## Chrysler Imperial 14.7   8 230 5.345  
## Duster 360      14.3   8 245 3.570  
## Camaro Z28      13.3   8 245 3.840  
## Cadillac Fleetwood 10.4   8 205 5.250  
## Lincoln Continental 10.4   8 215 5.424
```

8.2 Ejemplo 2: tuberías con iris

```
iris %>%  
  select(Sepal.Length, Petal.Length, Species) %>%  
  filter(Petal.Length > 4) %>%  
  arrange(desc(Petal.Length))
```

```
##   Sepal.Length Petal.Length   Species  
## 1           7.7           6.9 virginica  
## 2           7.7           6.7 virginica
```

## 3	7.7	6.7 virginica
## 4	7.6	6.6 virginica
## 5	7.9	6.4 virginica
## 6	7.3	6.3 virginica
## 7	7.2	6.1 virginica
## 8	7.4	6.1 virginica
## 9	7.7	6.1 virginica
## 10	6.3	6.0 virginica
## 11	7.2	6.0 virginica
## 12	7.1	5.9 virginica
## 13	6.8	5.9 virginica
## 14	6.5	5.8 virginica
## 15	6.7	5.8 virginica
## 16	7.2	5.8 virginica
## 17	6.9	5.7 virginica
## 18	6.7	5.7 virginica
## 19	6.7	5.7 virginica
## 20	6.3	5.6 virginica
## 21	6.4	5.6 virginica
## 22	6.4	5.6 virginica
## 23	6.1	5.6 virginica
## 24	6.3	5.6 virginica
## 25	6.7	5.6 virginica
## 26	6.8	5.5 virginica
## 27	6.5	5.5 virginica
## 28	6.4	5.5 virginica
## 29	6.9	5.4 virginica
## 30	6.2	5.4 virginica
## 31	6.4	5.3 virginica
## 32	6.4	5.3 virginica
## 33	6.7	5.2 virginica
## 34	6.5	5.2 virginica
## 35	6.0	5.1 versicolor
## 36	5.8	5.1 virginica
## 37	6.5	5.1 virginica
## 38	5.8	5.1 virginica
## 39	6.3	5.1 virginica
## 40	6.9	5.1 virginica
## 41	5.8	5.1 virginica
## 42	5.9	5.1 virginica
## 43	6.7	5.0 versicolor
## 44	5.7	5.0 virginica
## 45	6.0	5.0 virginica
## 46	6.3	5.0 virginica
## 47	6.9	4.9 versicolor
## 48	6.3	4.9 versicolor
## 49	5.6	4.9 virginica
## 50	6.3	4.9 virginica
## 51	6.1	4.9 virginica
## 52	5.9	4.8 versicolor
## 53	6.8	4.8 versicolor
## 54	6.2	4.8 virginica
## 55	6.0	4.8 virginica
## 56	7.0	4.7 versicolor

```
## 57      6.3      4.7 versicolor
## 58      6.1      4.7 versicolor
## 59      6.1      4.7 versicolor
## 60      6.7      4.7 versicolor
## 61      6.5      4.6 versicolor
## 62      6.6      4.6 versicolor
## 63      6.1      4.6 versicolor
## 64      6.4      4.5 versicolor
## 65      5.7      4.5 versicolor
## 66      5.6      4.5 versicolor
## 67      6.2      4.5 versicolor
## 68      6.0      4.5 versicolor
## 69      5.4      4.5 versicolor
## 70      6.0      4.5 versicolor
## 71      4.9      4.5 virginica
## 72      6.7      4.4 versicolor
## 73      6.6      4.4 versicolor
## 74      6.3      4.4 versicolor
## 75      5.5      4.4 versicolor
## 76      6.4      4.3 versicolor
## 77      6.2      4.3 versicolor
## 78      5.9      4.2 versicolor
## 79      5.6      4.2 versicolor
## 80      5.7      4.2 versicolor
## 81      5.7      4.2 versicolor
## 82      5.8      4.1 versicolor
## 83      5.6      4.1 versicolor
## 84      5.7      4.1 versicolor
```

8.3 Ejemplo 3: tuberías con airquality

```
airquality %>%
  na.omit() %>%
  select(Ozone, Wind, Temp, Month) %>%
  arrange(desc(Temp))
```

```
##      Ozone Wind Temp Month
## 1      76  9.7  97      8
## 2      84  6.3  96      8
## 3     118  2.3  94      8
## 4      85  6.3  94      8
## 5      73  2.8  93      9
## 6      91  4.6  93      9
## 7      97  6.3  92      7
## 8      97  5.7  92      7
## 9      78  5.1  92      9
## 10     96  6.9  91      9
## 11     71 13.8  90      6
## 12     89 10.3  90      8
```

## 13	110	8.0	90	8
## 14	85	7.4	89	7
## 15	122	4.0	89	8
## 16	77	5.1	88	7
## 17	82	7.4	88	7
## 18	39	11.5	87	6
## 19	79	5.1	87	7
## 20	47	7.4	87	9
## 21	80	8.6	86	7
## 22	52	12.0	86	7
## 23	50	7.4	86	7
## 24	44	11.5	86	8
## 25	73	8.0	86	8
## 26	49	9.2	85	7
## 27	63	11.5	85	7
## 28	108	8.0	85	7
## 29	135	4.1	84	7
## 30	61	6.3	84	7
## 31	32	15.5	84	9
## 32	64	4.6	83	7
## 33	40	10.9	83	7
## 34	64	7.4	83	7
## 35	29	9.7	82	6
## 36	23	8.0	82	6
## 37	35	10.3	82	7
## 38	20	8.6	82	7
## 39	16	7.4	82	8
## 40	28	11.5	82	8
## 41	16	8.0	82	9
## 42	45	14.9	81	5
## 43	32	9.2	81	7
## 44	27	14.9	81	7
## 45	48	6.9	81	7
## 46	59	9.2	81	7
## 47	39	6.9	81	8
## 48	9	13.8	81	8
## 49	168	3.4	81	8
## 50	44	14.9	81	9
## 51	36	10.3	81	9
## 52	7	14.3	80	7
## 53	65	9.7	80	8
## 54	20	10.9	80	9
## 55	115	5.7	79	5
## 56	59	6.3	79	8
## 57	45	9.7	79	8
## 58	31	10.9	78	8
## 59	44	10.3	78	8
## 60	23	10.3	78	9
## 61	46	6.9	78	9
## 62	21	14.9	77	6
## 63	22	10.3	77	8
## 64	21	15.5	77	8
## 65	28	6.3	77	9
## 66	37	7.4	76	5

## 67	13	10.3	76	6
## 68	23	7.4	76	8
## 69	21	15.5	76	9
## 70	13	10.3	76	9
## 71	18	8.0	76	9
## 72	21	10.9	75	9
## 73	14	14.3	75	9
## 74	12	12.6	74	5
## 75	16	6.9	74	7
## 76	11	16.6	73	5
## 77	12	11.5	73	6
## 78	10	14.3	73	7
## 79	24	9.7	73	9
## 80	36	8.0	72	5
## 81	37	20.7	72	6
## 82	9	14.3	72	8
## 83	9	10.9	71	9
## 84	13	11.5	71	9
## 85	23	9.2	71	9
## 86	30	6.9	70	9
## 87	16	9.7	69	5
## 88	7	10.3	69	9
## 89	14	10.9	68	5
## 90	30	11.5	68	5
## 91	24	10.3	68	9
## 92	20	11.5	68	9
## 93	41	7.4	67	5
## 94	23	12.0	67	5
## 95	18	13.8	67	9
## 96	11	9.2	66	5
## 97	34	12.0	66	5
## 98	23	8.6	65	5
## 99	20	9.2	65	6
## 100	14	11.5	64	5
## 101	13	12.6	64	9
## 102	14	16.6	63	9
## 103	18	11.5	62	5
## 104	11	9.7	62	5
## 105	8	20.1	61	5
## 106	4	9.7	61	5
## 107	32	12.0	61	5
## 108	19	13.8	59	5
## 109	1	9.7	59	5
## 110	18	13.2	58	5
## 111	6	18.4	57	5

9. Comparación de las cuatro formas

Ahora resolveremos el mismo problema usando las cuatro alternativas.

Queremos hacer lo siguiente:

1. Usar la base de datos `iris`.
2. Seleccionar las columnas `Sepal.Length`, `Sepal.Width`, `Petal.Length` y `Species`.
3. Filtrar las flores cuya longitud del pétalo sea mayor que 4.
4. Ordenar los resultados primero por `Species` y luego de mayor a menor según `Petal.Length`.

9.1 Forma 1: pasos intermedios

```
a1 <- iris
a2 <- select(a1, Sepal.Length, Sepal.Width, Petal.Length, Species)
a3 <- filter(a2, Petal.Length > 4)
a4 <- arrange(a3, Species, desc(Petal.Length))

a4
```

##	Sepal.Length	Sepal.Width	Petal.Length	Species
## 1	6.0	2.7	5.1	versicolor
## 2	6.7	3.0	5.0	versicolor
## 3	6.9	3.1	4.9	versicolor
## 4	6.3	2.5	4.9	versicolor
## 5	5.9	3.2	4.8	versicolor
## 6	6.8	2.8	4.8	versicolor
## 7	7.0	3.2	4.7	versicolor
## 8	6.3	3.3	4.7	versicolor
## 9	6.1	2.9	4.7	versicolor
## 10	6.1	2.8	4.7	versicolor
## 11	6.7	3.1	4.7	versicolor
## 12	6.5	2.8	4.6	versicolor
## 13	6.6	2.9	4.6	versicolor
## 14	6.1	3.0	4.6	versicolor
## 15	6.4	3.2	4.5	versicolor
## 16	5.7	2.8	4.5	versicolor
## 17	5.6	3.0	4.5	versicolor
## 18	6.2	2.2	4.5	versicolor
## 19	6.0	2.9	4.5	versicolor
## 20	5.4	3.0	4.5	versicolor
## 21	6.0	3.4	4.5	versicolor
## 22	6.7	3.1	4.4	versicolor
## 23	6.6	3.0	4.4	versicolor
## 24	6.3	2.3	4.4	versicolor
## 25	5.5	2.6	4.4	versicolor
## 26	6.4	2.9	4.3	versicolor
## 27	6.2	2.9	4.3	versicolor
## 28	5.9	3.0	4.2	versicolor
## 29	5.6	2.7	4.2	versicolor
## 30	5.7	3.0	4.2	versicolor
## 31	5.7	2.9	4.2	versicolor
## 32	5.8	2.7	4.1	versicolor
## 33	5.6	3.0	4.1	versicolor
## 34	5.7	2.8	4.1	versicolor
## 35	7.7	2.6	6.9	virginica

## 36	7.7	3.8	6.7	virginica
## 37	7.7	2.8	6.7	virginica
## 38	7.6	3.0	6.6	virginica
## 39	7.9	3.8	6.4	virginica
## 40	7.3	2.9	6.3	virginica
## 41	7.2	3.6	6.1	virginica
## 42	7.4	2.8	6.1	virginica
## 43	7.7	3.0	6.1	virginica
## 44	6.3	3.3	6.0	virginica
## 45	7.2	3.2	6.0	virginica
## 46	7.1	3.0	5.9	virginica
## 47	6.8	3.2	5.9	virginica
## 48	6.5	3.0	5.8	virginica
## 49	6.7	2.5	5.8	virginica
## 50	7.2	3.0	5.8	virginica
## 51	6.9	3.2	5.7	virginica
## 52	6.7	3.3	5.7	virginica
## 53	6.7	3.3	5.7	virginica
## 54	6.3	2.9	5.6	virginica
## 55	6.4	2.8	5.6	virginica
## 56	6.4	2.8	5.6	virginica
## 57	6.1	2.6	5.6	virginica
## 58	6.3	3.4	5.6	virginica
## 59	6.7	3.1	5.6	virginica
## 60	6.8	3.0	5.5	virginica
## 61	6.5	3.0	5.5	virginica
## 62	6.4	3.1	5.5	virginica
## 63	6.9	3.1	5.4	virginica
## 64	6.2	3.4	5.4	virginica
## 65	6.4	2.7	5.3	virginica
## 66	6.4	3.2	5.3	virginica
## 67	6.7	3.0	5.2	virginica
## 68	6.5	3.0	5.2	virginica
## 69	5.8	2.7	5.1	virginica
## 70	6.5	3.2	5.1	virginica
## 71	5.8	2.8	5.1	virginica
## 72	6.3	2.8	5.1	virginica
## 73	6.9	3.1	5.1	virginica
## 74	5.8	2.7	5.1	virginica
## 75	5.9	3.0	5.1	virginica
## 76	5.7	2.5	5.0	virginica
## 77	6.0	2.2	5.0	virginica
## 78	6.3	2.5	5.0	virginica
## 79	5.6	2.8	4.9	virginica
## 80	6.3	2.7	4.9	virginica
## 81	6.1	3.0	4.9	virginica
## 82	6.2	2.8	4.8	virginica
## 83	6.0	3.0	4.8	virginica
## 84	4.9	2.5	4.5	virginica

9.2 Forma 2: sobrescribir el objeto original

```
flores <- iris
flores <- select(flores, Sepal.Length, Sepal.Width, Petal.Length, Species)
flores <- filter(flores, Petal.Length > 4)
flores <- arrange(flores, Species, desc(Petal.Length))

flores
```

	Sepal.Length	Sepal.Width	Petal.Length	Species
## 1	6.0	2.7	5.1	versicolor
## 2	6.7	3.0	5.0	versicolor
## 3	6.9	3.1	4.9	versicolor
## 4	6.3	2.5	4.9	versicolor
## 5	5.9	3.2	4.8	versicolor
## 6	6.8	2.8	4.8	versicolor
## 7	7.0	3.2	4.7	versicolor
## 8	6.3	3.3	4.7	versicolor
## 9	6.1	2.9	4.7	versicolor
## 10	6.1	2.8	4.7	versicolor
## 11	6.7	3.1	4.7	versicolor
## 12	6.5	2.8	4.6	versicolor
## 13	6.6	2.9	4.6	versicolor
## 14	6.1	3.0	4.6	versicolor
## 15	6.4	3.2	4.5	versicolor
## 16	5.7	2.8	4.5	versicolor
## 17	5.6	3.0	4.5	versicolor
## 18	6.2	2.2	4.5	versicolor
## 19	6.0	2.9	4.5	versicolor
## 20	5.4	3.0	4.5	versicolor
## 21	6.0	3.4	4.5	versicolor
## 22	6.7	3.1	4.4	versicolor
## 23	6.6	3.0	4.4	versicolor
## 24	6.3	2.3	4.4	versicolor
## 25	5.5	2.6	4.4	versicolor
## 26	6.4	2.9	4.3	versicolor
## 27	6.2	2.9	4.3	versicolor
## 28	5.9	3.0	4.2	versicolor
## 29	5.6	2.7	4.2	versicolor
## 30	5.7	3.0	4.2	versicolor
## 31	5.7	2.9	4.2	versicolor
## 32	5.8	2.7	4.1	versicolor
## 33	5.6	3.0	4.1	versicolor
## 34	5.7	2.8	4.1	versicolor
## 35	7.7	2.6	6.9	virginica
## 36	7.7	3.8	6.7	virginica
## 37	7.7	2.8	6.7	virginica
## 38	7.6	3.0	6.6	virginica
## 39	7.9	3.8	6.4	virginica
## 40	7.3	2.9	6.3	virginica
## 41	7.2	3.6	6.1	virginica
## 42	7.4	2.8	6.1	virginica
## 43	7.7	3.0	6.1	virginica

## 44	6.3	3.3	6.0	virginica
## 45	7.2	3.2	6.0	virginica
## 46	7.1	3.0	5.9	virginica
## 47	6.8	3.2	5.9	virginica
## 48	6.5	3.0	5.8	virginica
## 49	6.7	2.5	5.8	virginica
## 50	7.2	3.0	5.8	virginica
## 51	6.9	3.2	5.7	virginica
## 52	6.7	3.3	5.7	virginica
## 53	6.7	3.3	5.7	virginica
## 54	6.3	2.9	5.6	virginica
## 55	6.4	2.8	5.6	virginica
## 56	6.4	2.8	5.6	virginica
## 57	6.1	2.6	5.6	virginica
## 58	6.3	3.4	5.6	virginica
## 59	6.7	3.1	5.6	virginica
## 60	6.8	3.0	5.5	virginica
## 61	6.5	3.0	5.5	virginica
## 62	6.4	3.1	5.5	virginica
## 63	6.9	3.1	5.4	virginica
## 64	6.2	3.4	5.4	virginica
## 65	6.4	2.7	5.3	virginica
## 66	6.4	3.2	5.3	virginica
## 67	6.7	3.0	5.2	virginica
## 68	6.5	3.0	5.2	virginica
## 69	5.8	2.7	5.1	virginica
## 70	6.5	3.2	5.1	virginica
## 71	5.8	2.8	5.1	virginica
## 72	6.3	2.8	5.1	virginica
## 73	6.9	3.1	5.1	virginica
## 74	5.8	2.7	5.1	virginica
## 75	5.9	3.0	5.1	virginica
## 76	5.7	2.5	5.0	virginica
## 77	6.0	2.2	5.0	virginica
## 78	6.3	2.5	5.0	virginica
## 79	5.6	2.8	4.9	virginica
## 80	6.3	2.7	4.9	virginica
## 81	6.1	3.0	4.9	virginica
## 82	6.2	2.8	4.8	virginica
## 83	6.0	3.0	4.8	virginica
## 84	4.9	2.5	4.5	virginica

9.3 Forma 3: composición de funciones

```

arrange(
  filter(
    select(iris, Sepal.Length, Sepal.Width, Petal.Length, Species),
    Petal.Length > 4
  ),
  Species,

```

```
desc(Petal.Length)
)
```

##	Sepal.Length	Sepal.Width	Petal.Length	Species
## 1	6.0	2.7	5.1	versicolor
## 2	6.7	3.0	5.0	versicolor
## 3	6.9	3.1	4.9	versicolor
## 4	6.3	2.5	4.9	versicolor
## 5	5.9	3.2	4.8	versicolor
## 6	6.8	2.8	4.8	versicolor
## 7	7.0	3.2	4.7	versicolor
## 8	6.3	3.3	4.7	versicolor
## 9	6.1	2.9	4.7	versicolor
## 10	6.1	2.8	4.7	versicolor
## 11	6.7	3.1	4.7	versicolor
## 12	6.5	2.8	4.6	versicolor
## 13	6.6	2.9	4.6	versicolor
## 14	6.1	3.0	4.6	versicolor
## 15	6.4	3.2	4.5	versicolor
## 16	5.7	2.8	4.5	versicolor
## 17	5.6	3.0	4.5	versicolor
## 18	6.2	2.2	4.5	versicolor
## 19	6.0	2.9	4.5	versicolor
## 20	5.4	3.0	4.5	versicolor
## 21	6.0	3.4	4.5	versicolor
## 22	6.7	3.1	4.4	versicolor
## 23	6.6	3.0	4.4	versicolor
## 24	6.3	2.3	4.4	versicolor
## 25	5.5	2.6	4.4	versicolor
## 26	6.4	2.9	4.3	versicolor
## 27	6.2	2.9	4.3	versicolor
## 28	5.9	3.0	4.2	versicolor
## 29	5.6	2.7	4.2	versicolor
## 30	5.7	3.0	4.2	versicolor
## 31	5.7	2.9	4.2	versicolor
## 32	5.8	2.7	4.1	versicolor
## 33	5.6	3.0	4.1	versicolor
## 34	5.7	2.8	4.1	versicolor
## 35	7.7	2.6	6.9	virginica
## 36	7.7	3.8	6.7	virginica
## 37	7.7	2.8	6.7	virginica
## 38	7.6	3.0	6.6	virginica
## 39	7.9	3.8	6.4	virginica
## 40	7.3	2.9	6.3	virginica
## 41	7.2	3.6	6.1	virginica
## 42	7.4	2.8	6.1	virginica
## 43	7.7	3.0	6.1	virginica
## 44	6.3	3.3	6.0	virginica
## 45	7.2	3.2	6.0	virginica
## 46	7.1	3.0	5.9	virginica
## 47	6.8	3.2	5.9	virginica
## 48	6.5	3.0	5.8	virginica
## 49	6.7	2.5	5.8	virginica

## 50	7.2	3.0	5.8	virginica
## 51	6.9	3.2	5.7	virginica
## 52	6.7	3.3	5.7	virginica
## 53	6.7	3.3	5.7	virginica
## 54	6.3	2.9	5.6	virginica
## 55	6.4	2.8	5.6	virginica
## 56	6.4	2.8	5.6	virginica
## 57	6.1	2.6	5.6	virginica
## 58	6.3	3.4	5.6	virginica
## 59	6.7	3.1	5.6	virginica
## 60	6.8	3.0	5.5	virginica
## 61	6.5	3.0	5.5	virginica
## 62	6.4	3.1	5.5	virginica
## 63	6.9	3.1	5.4	virginica
## 64	6.2	3.4	5.4	virginica
## 65	6.4	2.7	5.3	virginica
## 66	6.4	3.2	5.3	virginica
## 67	6.7	3.0	5.2	virginica
## 68	6.5	3.0	5.2	virginica
## 69	5.8	2.7	5.1	virginica
## 70	6.5	3.2	5.1	virginica
## 71	5.8	2.8	5.1	virginica
## 72	6.3	2.8	5.1	virginica
## 73	6.9	3.1	5.1	virginica
## 74	5.8	2.7	5.1	virginica
## 75	5.9	3.0	5.1	virginica
## 76	5.7	2.5	5.0	virginica
## 77	6.0	2.2	5.0	virginica
## 78	6.3	2.5	5.0	virginica
## 79	5.6	2.8	4.9	virginica
## 80	6.3	2.7	4.9	virginica
## 81	6.1	3.0	4.9	virginica
## 82	6.2	2.8	4.8	virginica
## 83	6.0	3.0	4.8	virginica
## 84	4.9	2.5	4.5	virginica

9.4 Forma 4: tuberías

```
iris %>%
  select(Sepal.Length, Sepal.Width, Petal.Length, Species) %>%
  filter(Petal.Length > 4) %>%
  arrange(Species, desc(Petal.Length))
```

##	Sepal.Length	Sepal.Width	Petal.Length	Species
## 1	6.0	2.7	5.1	versicolor
## 2	6.7	3.0	5.0	versicolor
## 3	6.9	3.1	4.9	versicolor
## 4	6.3	2.5	4.9	versicolor
## 5	5.9	3.2	4.8	versicolor

## 6	6.8	2.8	4.8 versicolor
## 7	7.0	3.2	4.7 versicolor
## 8	6.3	3.3	4.7 versicolor
## 9	6.1	2.9	4.7 versicolor
## 10	6.1	2.8	4.7 versicolor
## 11	6.7	3.1	4.7 versicolor
## 12	6.5	2.8	4.6 versicolor
## 13	6.6	2.9	4.6 versicolor
## 14	6.1	3.0	4.6 versicolor
## 15	6.4	3.2	4.5 versicolor
## 16	5.7	2.8	4.5 versicolor
## 17	5.6	3.0	4.5 versicolor
## 18	6.2	2.2	4.5 versicolor
## 19	6.0	2.9	4.5 versicolor
## 20	5.4	3.0	4.5 versicolor
## 21	6.0	3.4	4.5 versicolor
## 22	6.7	3.1	4.4 versicolor
## 23	6.6	3.0	4.4 versicolor
## 24	6.3	2.3	4.4 versicolor
## 25	5.5	2.6	4.4 versicolor
## 26	6.4	2.9	4.3 versicolor
## 27	6.2	2.9	4.3 versicolor
## 28	5.9	3.0	4.2 versicolor
## 29	5.6	2.7	4.2 versicolor
## 30	5.7	3.0	4.2 versicolor
## 31	5.7	2.9	4.2 versicolor
## 32	5.8	2.7	4.1 versicolor
## 33	5.6	3.0	4.1 versicolor
## 34	5.7	2.8	4.1 versicolor
## 35	7.7	2.6	6.9 virginica
## 36	7.7	3.8	6.7 virginica
## 37	7.7	2.8	6.7 virginica
## 38	7.6	3.0	6.6 virginica
## 39	7.9	3.8	6.4 virginica
## 40	7.3	2.9	6.3 virginica
## 41	7.2	3.6	6.1 virginica
## 42	7.4	2.8	6.1 virginica
## 43	7.7	3.0	6.1 virginica
## 44	6.3	3.3	6.0 virginica
## 45	7.2	3.2	6.0 virginica
## 46	7.1	3.0	5.9 virginica
## 47	6.8	3.2	5.9 virginica
## 48	6.5	3.0	5.8 virginica
## 49	6.7	2.5	5.8 virginica
## 50	7.2	3.0	5.8 virginica
## 51	6.9	3.2	5.7 virginica
## 52	6.7	3.3	5.7 virginica
## 53	6.7	3.3	5.7 virginica
## 54	6.3	2.9	5.6 virginica
## 55	6.4	2.8	5.6 virginica
## 56	6.4	2.8	5.6 virginica
## 57	6.1	2.6	5.6 virginica
## 58	6.3	3.4	5.6 virginica
## 59	6.7	3.1	5.6 virginica

```
## 60      6.8      3.0      5.5 virginica
## 61      6.5      3.0      5.5 virginica
## 62      6.4      3.1      5.5 virginica
## 63      6.9      3.1      5.4 virginica
## 64      6.2      3.4      5.4 virginica
## 65      6.4      2.7      5.3 virginica
## 66      6.4      3.2      5.3 virginica
## 67      6.7      3.0      5.2 virginica
## 68      6.5      3.0      5.2 virginica
## 69      5.8      2.7      5.1 virginica
## 70      6.5      3.2      5.1 virginica
## 71      5.8      2.8      5.1 virginica
## 72      6.3      2.8      5.1 virginica
## 73      6.9      3.1      5.1 virginica
## 74      5.8      2.7      5.1 virginica
## 75      5.9      3.0      5.1 virginica
## 76      5.7      2.5      5.0 virginica
## 77      6.0      2.2      5.0 virginica
## 78      6.3      2.5      5.0 virginica
## 79      5.6      2.8      4.9 virginica
## 80      6.3      2.7      4.9 virginica
## 81      6.1      3.0      4.9 virginica
## 82      6.2      2.8      4.8 virginica
## 83      6.0      3.0      4.8 virginica
## 84      4.9      2.5      4.5 virginica
```

La cuarta forma suele ser la más fácil de leer, porque las instrucciones aparecen en el mismo orden en que pensamos el análisis:

1. Tomar la base de datos.
2. Seleccionar columnas.
3. Filtrar filas.
4. Ordenar resultados.

10. ¿Cómo funciona la tubería?

La tubería `%>%` toma lo que está a la izquierda y lo coloca como primer argumento de la función que está a la derecha.

Por ejemplo:

```
mtcars %>%
  head()
```

```
##      mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710     22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive  21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
## Valiant        18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

es equivalente a:

```
head(mtcars)
```

```
##           mpg cyl  disp  hp  drat    wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160 110 3.90 2.620 16.46 0  1    4    4
## Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02 0  1    4    4
## Datsun 710     22.8   4  108  93 3.85 2.320 18.61 1  1    4    1
## Hornet 4 Drive  21.4   6  258 110 3.08 3.215 19.44 1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02 0  0    3    2
## Valiant        18.1   6  225 105 2.76 3.460 20.22 1  0    3    1
```

Otro ejemplo:

```
mtcars %>%
  select(mpg, cyl, hp)
```

```
##           mpg cyl  hp
## Mazda RX4      21.0   6 110
## Mazda RX4 Wag  21.0   6 110
## Datsun 710     22.8   4  93
## Hornet 4 Drive  21.4   6 110
## Hornet Sportabout 18.7   8 175
## Valiant        18.1   6 105
## Duster 360     14.3   8 245
## Merc 240D      24.4   4  62
## Merc 230       22.8   4  95
## Merc 280       19.2   6 123
## Merc 280C      17.8   6 123
## Merc 450SE     16.4   8 180
## Merc 450SL     17.3   8 180
## Merc 450SLC    15.2   8 180
## Cadillac Fleetwood 10.4   8 205
## Lincoln Continental 10.4   8 215
## Chrysler Imperial 14.7   8 230
## Fiat 128       32.4   4  66
## Honda Civic    30.4   4  52
## Toyota Corolla 33.9   4  65
## Toyota Corona  21.5   4  97
## Dodge Challenger 15.5   8 150
## AMC Javelin    15.2   8 150
## Camaro Z28     13.3   8 245
## Pontiac Firebird 19.2   8 175
## Fiat X1-9      27.3   4  66
## Porsche 914-2  26.0   4  91
## Lotus Europa   30.4   4 113
## Ford Pantera L 15.8   8 264
## Ferrari Dino   19.7   6 175
## Maserati Bora   15.0   8 335
## Volvo 142E     21.4   4 109
```

es equivalente a:

```
select(mtcars, mpg, cyl, hp)
```

```
##           mpg cyl  hp
## Mazda RX4      21.0   6 110
## Mazda RX4 Wag  21.0   6 110
## Datsun 710      22.8   4  93
## Hornet 4 Drive  21.4   6 110
## Hornet Sportabout 18.7   8 175
## Valiant         18.1   6 105
## Duster 360      14.3   8 245
## Merc 240D       24.4   4  62
## Merc 230        22.8   4  95
## Merc 280        19.2   6 123
## Merc 280C       17.8   6 123
## Merc 450SE      16.4   8 180
## Merc 450SL      17.3   8 180
## Merc 450SLC     15.2   8 180
## Cadillac Fleetwood 10.4   8 205
## Lincoln Continental 10.4   8 215
## Chrysler Imperial 14.7   8 230
## Fiat 128        32.4   4  66
## Honda Civic     30.4   4  52
## Toyota Corolla  33.9   4  65
## Toyota Corona   21.5   4  97
## Dodge Challenger 15.5   8 150
## AMC Javelin     15.2   8 150
## Camaro Z28      13.3   8 245
## Pontiac Firebird 19.2   8 175
## Fiat X1-9       27.3   4  66
## Porsche 914-2   26.0   4  91
## Lotus Europa    30.4   4 113
## Ford Pantera L  15.8   8 264
## Ferrari Dino    19.7   6 175
## Maserati Bora   15.0   8 335
## Volvo 142E      21.4   4 109
```

Es decir, la tubería evita que tengamos que escribir muchas veces el nombre del objeto.

10.1 Ejemplo 1: sin tubería

```
select(mtcars, mpg, cyl, hp)
```

```
##           mpg cyl  hp
## Mazda RX4      21.0   6 110
## Mazda RX4 Wag  21.0   6 110
## Datsun 710      22.8   4  93
## Hornet 4 Drive  21.4   6 110
## Hornet Sportabout 18.7   8 175
```

## Valiant	18.1	6	105
## Duster 360	14.3	8	245
## Merc 240D	24.4	4	62
## Merc 230	22.8	4	95
## Merc 280	19.2	6	123
## Merc 280C	17.8	6	123
## Merc 450SE	16.4	8	180
## Merc 450SL	17.3	8	180
## Merc 450SLC	15.2	8	180
## Cadillac Fleetwood	10.4	8	205
## Lincoln Continental	10.4	8	215
## Chrysler Imperial	14.7	8	230
## Fiat 128	32.4	4	66
## Honda Civic	30.4	4	52
## Toyota Corolla	33.9	4	65
## Toyota Corona	21.5	4	97
## Dodge Challenger	15.5	8	150
## AMC Javelin	15.2	8	150
## Camaro Z28	13.3	8	245
## Pontiac Firebird	19.2	8	175
## Fiat X1-9	27.3	4	66
## Porsche 914-2	26.0	4	91
## Lotus Europa	30.4	4	113
## Ford Pantera L	15.8	8	264
## Ferrari Dino	19.7	6	175
## Maserati Bora	15.0	8	335
## Volvo 142E	21.4	4	109

10.2 Ejemplo 2: con tubería

```
mtcars %>%
  select(mpg, cyl, hp)
```

##	mpg	cyl	hp
## Mazda RX4	21.0	6	110
## Mazda RX4 Wag	21.0	6	110
## Datsun 710	22.8	4	93
## Hornet 4 Drive	21.4	6	110
## Hornet Sportabout	18.7	8	175
## Valiant	18.1	6	105
## Duster 360	14.3	8	245
## Merc 240D	24.4	4	62
## Merc 230	22.8	4	95
## Merc 280	19.2	6	123
## Merc 280C	17.8	6	123
## Merc 450SE	16.4	8	180
## Merc 450SL	17.3	8	180
## Merc 450SLC	15.2	8	180
## Cadillac Fleetwood	10.4	8	205

```
## Lincoln Continental 10.4 8 215
## Chrysler Imperial 14.7 8 230
## Fiat 128 32.4 4 66
## Honda Civic 30.4 4 52
## Toyota Corolla 33.9 4 65
## Toyota Corona 21.5 4 97
## Dodge Challenger 15.5 8 150
## AMC Javelin 15.2 8 150
## Camaro Z28 13.3 8 245
## Pontiac Firebird 19.2 8 175
## Fiat X1-9 27.3 4 66
## Porsche 914-2 26.0 4 91
## Lotus Europa 30.4 4 113
## Ford Pantera L 15.8 8 264
## Ferrari Dino 19.7 6 175
## Maserati Bora 15.0 8 335
## Volvo 142E 21.4 4 109
```

11. Funciones que usaremos con tuberías

Aunque esta clase se enfoca en las alternativas a las tuberías, usaremos algunas funciones importantes de `dplyr`.

Las principales son:

- `select()`: seleccionar columnas.
 - `filter()`: filtrar filas.
 - `mutate()`: crear nuevas columnas.
 - `arrange()`: ordenar filas.
 - `group_by()`: agrupar datos.
 - `summarise()`: resumir información.
 - `na.omit()`: eliminar valores faltantes.
-

12. Ejemplo completo de clase

Supongamos que queremos trabajar con la base `mtcars`.

Queremos:

1. Seleccionar las columnas `mpg`, `cyl`, `hp`, `wt` y `gear`.
2. Filtrar autos con más de 100 caballos de fuerza.
3. Crear una nueva columna llamada `eficiencia`.
4. Agrupar por número de cilindros.
5. Calcular el promedio de eficiencia.
6. Ordenar los resultados de mayor a menor.

```
mtcars %>%
  select(mpg, cyl, hp, wt, gear) %>%
  filter(hp > 100) %>%
  mutate(eficiencia = mpg / wt) %>%
  group_by(cyl) %>%
  summarise(
    promedio_eficiencia = mean(eficiencia),
    cantidad_autos = n()
  ) %>%
  arrange(desc(promedio_eficiencia))
```

```
## # A tibble: 3 x 3
##   cyl promedio_eficiencia cantidad_autos
##   <dbl>                <dbl>         <int>
## 1     4                 13.9             2
## 2     6                  6.44             7
## 3     8                  3.95            14
```

El flujo de trabajo fue:

```
Base de datos
↓
select()
↓
filter()
↓
mutate()
↓
group_by()
↓
summarise()
↓
arrange()
```

14. Ejercicios para practicar el primer ítem

Ejercicio 1

Usa la base de datos `mtcars`.

Realiza lo siguiente usando **pasos intermedios**:

1. Selecciona las columnas `mpg`, `cyl`, `hp` y `wt`.
2. Filtra los automóviles con `hp > 100`.
3. Ordena los resultados de mayor a menor según `mpg`.

```
library(dplyr)
```

```
e1 <- mtcars  
e1 <- select(e1, mpg, cyl, hp, wt)  
e1 <- filter(e1, hp > 100)  
e1 <- arrange(e1, desc(mpg))
```

```
e1
```

```
##           mpg  cyl  hp   wt  
## Lotus Europa      30.4   4 113 1.513  
## Hornet 4 Drive    21.4   6 110 3.215  
## Volvo 142E        21.4   4 109 2.780  
## Mazda RX4         21.0   6 110 2.620  
## Mazda RX4 Wag     21.0   6 110 2.875  
## Ferrari Dino      19.7   6 175 2.770  
## Merc 280          19.2   6 123 3.440  
## Pontiac Firebird  19.2   8 175 3.845  
## Hornet Sportabout 18.7   8 175 3.440  
## Valiant           18.1   6 105 3.460  
## Merc 280C         17.8   6 123 3.440  
## Merc 450SL        17.3   8 180 3.730  
## Merc 450SE        16.4   8 180 4.070  
## Ford Pantera L    15.8   8 264 3.170  
## Dodge Challenger  15.5   8 150 3.520  
## Merc 450SLC       15.2   8 180 3.780  
## AMC Javelin       15.2   8 150 3.435  
## Maserati Bora     15.0   8 335 3.570  
## Chrysler Imperial 14.7   8 230 5.345  
## Duster 360        14.3   8 245 3.570  
## Camaro Z28        13.3   8 245 3.840  
## Cadillac Fleetwood 10.4   8 205 5.250  
## Lincoln Continental 10.4   8 215 5.424
```

Ejercicio 2

Usa la base de datos `iris`.

Realiza lo siguiente usando **pasos intermedios**:

1. Selecciona las columnas `Sepal.Length`, `Petal.Length` y `Species`.
2. Filtra las flores con `Petal.Length > 4`.
3. Ordena los resultados de mayor a menor según `Petal.Length`.

```
e2 <- iris  
e2 <- select(e2, Sepal.Length, Petal.Length, Species)  
e2 <- filter(e2, Petal.Length > 4)  
e2 <- arrange(e2, desc(Petal.Length))
```

```
e2
```

##	Sepal.Length	Petal.Length	Species
## 1	7.7	6.9	virginica
## 2	7.7	6.7	virginica
## 3	7.7	6.7	virginica
## 4	7.6	6.6	virginica
## 5	7.9	6.4	virginica
## 6	7.3	6.3	virginica
## 7	7.2	6.1	virginica
## 8	7.4	6.1	virginica
## 9	7.7	6.1	virginica
## 10	6.3	6.0	virginica
## 11	7.2	6.0	virginica
## 12	7.1	5.9	virginica
## 13	6.8	5.9	virginica
## 14	6.5	5.8	virginica
## 15	6.7	5.8	virginica
## 16	7.2	5.8	virginica
## 17	6.9	5.7	virginica
## 18	6.7	5.7	virginica
## 19	6.7	5.7	virginica
## 20	6.3	5.6	virginica
## 21	6.4	5.6	virginica
## 22	6.4	5.6	virginica
## 23	6.1	5.6	virginica
## 24	6.3	5.6	virginica
## 25	6.7	5.6	virginica
## 26	6.8	5.5	virginica
## 27	6.5	5.5	virginica
## 28	6.4	5.5	virginica
## 29	6.9	5.4	virginica
## 30	6.2	5.4	virginica
## 31	6.4	5.3	virginica
## 32	6.4	5.3	virginica
## 33	6.7	5.2	virginica
## 34	6.5	5.2	virginica
## 35	6.0	5.1	versicolor
## 36	5.8	5.1	virginica
## 37	6.5	5.1	virginica
## 38	5.8	5.1	virginica
## 39	6.3	5.1	virginica
## 40	6.9	5.1	virginica
## 41	5.8	5.1	virginica
## 42	5.9	5.1	virginica
## 43	6.7	5.0	versicolor
## 44	5.7	5.0	virginica
## 45	6.0	5.0	virginica
## 46	6.3	5.0	virginica
## 47	6.9	4.9	versicolor
## 48	6.3	4.9	versicolor
## 49	5.6	4.9	virginica
## 50	6.3	4.9	virginica
## 51	6.1	4.9	virginica
## 52	5.9	4.8	versicolor
## 53	6.8	4.8	versicolor

```
## 54      6.2      4.8 virginica
## 55      6.0      4.8 virginica
## 56      7.0      4.7 versicolor
## 57      6.3      4.7 versicolor
## 58      6.1      4.7 versicolor
## 59      6.1      4.7 versicolor
## 60      6.7      4.7 versicolor
## 61      6.5      4.6 versicolor
## 62      6.6      4.6 versicolor
## 63      6.1      4.6 versicolor
## 64      6.4      4.5 versicolor
## 65      5.7      4.5 versicolor
## 66      5.6      4.5 versicolor
## 67      6.2      4.5 versicolor
## 68      6.0      4.5 versicolor
## 69      5.4      4.5 versicolor
## 70      6.0      4.5 versicolor
## 71      4.9      4.5 virginica
## 72      6.7      4.4 versicolor
## 73      6.6      4.4 versicolor
## 74      6.3      4.4 versicolor
## 75      5.5      4.4 versicolor
## 76      6.4      4.3 versicolor
## 77      6.2      4.3 versicolor
## 78      5.9      4.2 versicolor
## 79      5.6      4.2 versicolor
## 80      5.7      4.2 versicolor
## 81      5.7      4.2 versicolor
## 82      5.8      4.1 versicolor
## 83      5.6      4.1 versicolor
## 84      5.7      4.1 versicolor
```

Ejercicio 3

Usa la base de datos `ToothGrowth`.

Realiza lo siguiente **sobrescribiendo el objeto original**:

1. Guarda `ToothGrowth` en un objeto llamado `dientes`.
2. Selecciona las columnas `len`, `supp` y `dose`.
3. Filtra los registros donde `dose >= 1`.
4. Ordena los resultados primero por `supp` y luego de mayor a menor por `len`.

```
dientes <- ToothGrowth
dientes <- select(dientes, len, supp, dose)
dientes <- filter(dientes, dose >= 1)
dientes <- arrange(dientes, supp, desc(len))

dientes
```

```
##      len supp dose
```

##	1	30.9	OJ	2
##	2	29.4	OJ	2
##	3	27.3	OJ	1
##	4	27.3	OJ	2
##	5	26.4	OJ	1
##	6	26.4	OJ	2
##	7	26.4	OJ	2
##	8	25.8	OJ	1
##	9	25.5	OJ	2
##	10	25.2	OJ	1
##	11	24.8	OJ	2
##	12	24.5	OJ	2
##	13	23.6	OJ	1
##	14	23.3	OJ	1
##	15	23.0	OJ	2
##	16	22.4	OJ	2
##	17	21.2	OJ	1
##	18	20.0	OJ	1
##	19	19.7	OJ	1
##	20	14.5	OJ	1
##	21	33.9	VC	2
##	22	32.5	VC	2
##	23	29.5	VC	2
##	24	26.7	VC	2
##	25	26.4	VC	2
##	26	25.5	VC	2
##	27	23.6	VC	2
##	28	23.3	VC	2
##	29	22.5	VC	1
##	30	21.5	VC	2
##	31	18.8	VC	1
##	32	18.5	VC	2
##	33	17.3	VC	1
##	34	17.3	VC	1
##	35	16.5	VC	1
##	36	16.5	VC	1
##	37	15.5	VC	1
##	38	15.2	VC	1
##	39	14.5	VC	1
##	40	13.6	VC	1

Ejercicio 4

Usa la base de datos `airquality`.

Realiza lo siguiente usando **composición de funciones**:

1. Elimina los valores faltantes.
2. Selecciona las columnas `Ozone`, `Wind`, `Temp` y `Month`.
3. Ordena los datos de mayor a menor según `Temp`.

```

arrange(
  select(
    na.omit(airquality),
    Ozone, Wind, Temp, Month
  ),
  desc(Temp)
)

```

```

##      Ozone Wind Temp Month
## 1      76  9.7  97      8
## 2      84  6.3  96      8
## 3     118  2.3  94      8
## 4      85  6.3  94      8
## 5      73  2.8  93      9
## 6      91  4.6  93      9
## 7      97  6.3  92      7
## 8      97  5.7  92      7
## 9      78  5.1  92      9
## 10     96  6.9  91      9
## 11     71 13.8  90      6
## 12     89 10.3  90      8
## 13    110  8.0  90      8
## 14     85  7.4  89      7
## 15    122  4.0  89      8
## 16     77  5.1  88      7
## 17     82  7.4  88      7
## 18     39 11.5  87      6
## 19     79  5.1  87      7
## 20     47  7.4  87      9
## 21     80  8.6  86      7
## 22     52 12.0  86      7
## 23     50  7.4  86      7
## 24     44 11.5  86      8
## 25     73  8.0  86      8
## 26     49  9.2  85      7
## 27     63 11.5  85      7
## 28    108  8.0  85      7
## 29    135  4.1  84      7
## 30     61  6.3  84      7
## 31     32 15.5  84      9
## 32     64  4.6  83      7
## 33     40 10.9  83      7
## 34     64  7.4  83      7
## 35     29  9.7  82      6
## 36     23  8.0  82      6
## 37     35 10.3  82      7
## 38     20  8.6  82      7
## 39     16  7.4  82      8
## 40     28 11.5  82      8
## 41     16  8.0  82      9
## 42     45 14.9  81      5
## 43     32  9.2  81      7
## 44     27 14.9  81      7

```

## 45	48	6.9	81	7
## 46	59	9.2	81	7
## 47	39	6.9	81	8
## 48	9	13.8	81	8
## 49	168	3.4	81	8
## 50	44	14.9	81	9
## 51	36	10.3	81	9
## 52	7	14.3	80	7
## 53	65	9.7	80	8
## 54	20	10.9	80	9
## 55	115	5.7	79	5
## 56	59	6.3	79	8
## 57	45	9.7	79	8
## 58	31	10.9	78	8
## 59	44	10.3	78	8
## 60	23	10.3	78	9
## 61	46	6.9	78	9
## 62	21	14.9	77	6
## 63	22	10.3	77	8
## 64	21	15.5	77	8
## 65	28	6.3	77	9
## 66	37	7.4	76	5
## 67	13	10.3	76	6
## 68	23	7.4	76	8
## 69	21	15.5	76	9
## 70	13	10.3	76	9
## 71	18	8.0	76	9
## 72	21	10.9	75	9
## 73	14	14.3	75	9
## 74	12	12.6	74	5
## 75	16	6.9	74	7
## 76	11	16.6	73	5
## 77	12	11.5	73	6
## 78	10	14.3	73	7
## 79	24	9.7	73	9
## 80	36	8.0	72	5
## 81	37	20.7	72	6
## 82	9	14.3	72	8
## 83	9	10.9	71	9
## 84	13	11.5	71	9
## 85	23	9.2	71	9
## 86	30	6.9	70	9
## 87	16	9.7	69	5
## 88	7	10.3	69	9
## 89	14	10.9	68	5
## 90	30	11.5	68	5
## 91	24	10.3	68	9
## 92	20	11.5	68	9
## 93	41	7.4	67	5
## 94	23	12.0	67	5
## 95	18	13.8	67	9
## 96	11	9.2	66	5
## 97	34	12.0	66	5
## 98	23	8.6	65	5

```
## 99      20  9.2   65    6
## 100     14 11.5   64    5
## 101     13 12.6   64    9
## 102     14 16.6   63    9
## 103     18 11.5   62    5
## 104     11  9.7   62    5
## 105      8 20.1   61    5
## 106      4  9.7   61    5
## 107     32 12.0   61    5
## 108     19 13.8   59    5
## 109      1  9.7   59    5
## 110     18 13.2   58    5
## 111      6 18.4   57    5
```

Ejercicio 5

Usa la base de datos `mtcars`.

Realiza lo siguiente usando **tuberías**:

1. Selecciona las columnas `mpg`, `cyl`, `hp` y `wt`.
2. Filtra los automóviles con `cyl == 6`.
3. Ordena los resultados de mayor a menor según `hp`.

```
mtcars %>%
  select(mpg, cyl, hp, wt) %>%
  filter(cyl == 6) %>%
  arrange(desc(hp))
```

```
##           mpg cyl  hp  wt
## Ferrari Dino 19.7  6 175 2.770
## Merc 280     19.2  6 123 3.440
## Merc 280C    17.8  6 123 3.440
## Mazda RX4    21.0  6 110 2.620
## Mazda RX4 Wag 21.0  6 110 2.875
## Hornet 4 Drive 21.4  6 110 3.215
## Valiant      18.1  6 105 3.460
```

Ejercicio 6

Usa la base de datos `mtcars`.

Realiza lo siguiente usando tuberías:

1. Selecciona las columnas `mpg`, `cyl`, `hp`, `wt` y `gear`.
2. Filtra los automóviles con `hp > 100`.
3. Crea una nueva variable llamada `eficiencia` definida como:

$$eficiencia = \frac{mpg}{wt}$$

4. Agrupa por `cyl`.
5. Calcula:
 - promedio de `eficiencia`
 - cantidad de automóviles
6. Ordena los resultados de mayor a menor según el promedio de eficiencia.

```
mtcars %>%
  select(mpg, cyl, hp, wt, gear) %>%
  filter(hp > 100) %>%
  mutate(eficiencia = mpg / wt) %>%
  group_by(cyl) %>%
  summarise(
    promedio_eficiencia = mean(eficiencia),
    n = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(promedio_eficiencia))
```

```
## # A tibble: 3 x 3
##   cyl promedio_eficiencia      n
##   <dbl>                <dbl> <int>
## 1     4                 13.9     2
## 2     6                 6.44     7
## 3     8                 3.95    14
```

Ejercicio 7

Usa la base de datos `iris`.

Realiza lo siguiente usando tuberías:

1. Selecciona las columnas `Sepal.Length`, `Sepal.Width`, `Petal.Length`, `Petal.Width` y `Species`.
2. Filtra las flores con `Petal.Length > 3`.
3. Crea una variable llamada `relacion_petal`:

$$relacion_petalo = \frac{Petal.Length}{Petal.Width}$$

4. Agrupa por `Species`.
 5. Calcula:
 - promedio de `relacion_petal`
 - máximo de `Sepal.Length`
 - cantidad de observaciones
 6. Ordena los resultados de mayor a menor según el promedio de `relacion_petal`.
-

Ejercicio 8

Usa la base de datos `airquality`.

Realiza lo siguiente usando tuberías:

1. Elimina los valores faltantes.
2. Selecciona las columnas `Ozone`, `Wind`, `Temp` y `Month`.
3. Filtra los días donde `Temp > 80`.
4. Crea una variable llamada `indice_ozono`:

$$\text{indice_ozono} = \frac{\text{Ozone}}{\text{Wind}}$$

5. Agrupa por `Month`.
6. Calcula:
 - promedio de `indice_ozono`
 - temperatura promedio
 - cantidad de observaciones
7. Ordena los resultados de mayor a menor según el promedio del índice de ozono.

```
airquality %>%
  na.omit() %>%
  select(Ozone, Wind, Temp, Month) %>%
  filter(Temp > 80) %>%
  mutate(indice_ozono = Ozone / Wind) %>%
  group_by(Month) %>%
  summarise(
    promedio_indice = mean(indice_ozono),
    temp_promedio = mean(Temp),
    n = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(promedio_indice))
```

```
## # A tibble: 5 x 4
##   Month promedio_indice temp_promedio     n
##   <int>          <dbl>          <dbl> <int>
## 1     8           15.2           87.8    14
## 2     9           10.2           87.1     9
## 3     7            9.72          85.0    23
## 4     6            3.60          85.2     4
## 5     5            3.02           81     1
```

Ejercicio 9

Usa la base de datos `ToothGrowth`.

Realiza lo siguiente usando tuberías:

1. Selecciona las columnas `len`, `supp` y `dose`.
2. Filtra los datos donde `dose >= 1`.
3. Crea una variable llamada `log_len`:

$$\log_len = \log(len)$$

4. Agrupa por `supp` y `dose`.
5. Calcula:
 - promedio de `log_len`
 - desviación estándar de `len`
 - cantidad de observaciones
6. Ordena primero por `supp` y luego de mayor a menor según el promedio de `log_len`.

```

ToothGrowth %>%
  select(len, supp, dose) %>%
  filter(dose >= 1) %>%
  mutate(log_len = log(len)) %>%
  group_by(supp, dose) %>%
  summarise(
    promedio_log = mean(log_len),
    sd_len = sd(len),
    n = n(),
    .groups = "drop"
  ) %>%
  arrange(supp, desc(promedio_log))

```

```

## # A tibble: 4 x 5
##   supp  dose promedio_log sd_len    n
##   <fct> <dbl>         <dbl> <dbl> <int>
## 1 OJ      2         3.26   2.66   10
## 2 OJ      1         3.11   3.91   10
## 3 VC      2         3.25   4.80   10
## 4 VC      1         2.81   2.52   10

```

Ejercicio 10

Usa la base de datos `mtcars`.

Este ejercicio combina varias herramientas.

Realiza lo siguiente:

1. Selecciona las columnas `mpg`, `cyl`, `hp`, `wt`, `gear` y `am`.
2. Filtra los automóviles que cumplan:
 - `hp > 100`
 - `wt < 4`
3. Crea dos nuevas variables:

$$potencia_relativa = \frac{hp}{wt}$$

$$consumo_inverso = \frac{1}{mpg}$$

4. Agrupa por `gear` y `am`.

5. Calcula:

- promedio de `potencia_relativa`
- promedio de `consumo_inverso`
- cantidad de observaciones

6. Ordena los resultados:

- primero de mayor a menor según `promedio_potencia_relativa`
- luego de menor a mayor según `promedio_consumo_inverso`.

```
mtcars %>%
  select(mpg, cyl, hp, wt, gear, am) %>%
  filter(hp > 100, wt < 4) %>%
  mutate(
    potencia_relativa = hp / wt,
    consumo_inverso = 1 / mpg
  ) %>%
  group_by(gear, am) %>%
  summarise(
    promedio_potencia = mean(potencia_relativa),
    promedio_consumo = mean(consumo_inverso),
    n = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(promedio_potencia), promedio_consumo)
```

```
## # A tibble: 4 x 5
##   gear    am promedio_potencia promedio_consumo     n
##   <dbl> <dbl>           <dbl>           <dbl> <int>
## 1     5     1             78.7             0.0534     4
## 2     3     0             47.6             0.0607    10
## 3     4     1             39.8             0.0473     3
## 4     4     0             35.8             0.0541     2
```

16. Comentario final

En esta clase estudiamos la introducción a las tuberías con el paquete `magrittr`.

Vimos que una misma tarea puede escribirse de varias formas:

- Guardando pasos intermedios.
- Sobrescribiendo el objeto original.

- Componiendo funciones.
- Usando tuberías.

La tubería `%>%` suele ser muy útil cuando tenemos una secuencia lineal de operaciones.